Object Placement Localization in Street Scenes: A Candidate Ranking Framework with Support-Surface Constraints

Qihang Feng
University of Alberta
Edmonton, Canada
qfeng5@ualberta.ca

Zixi Chai
University of Alberta
Edmonton, Canada
zchai3@ualberta.ca

Hongtao Ren
University of Alberta
Edmonton, Canada
hren7@ualberta.ca

Abstract

Object placement in street scenes requires determining the position of a given object in the image, but direct regression boundary box coordinates tend to degrade to an average position when there are multiple valid positions. In this study, we adopt the candidate ranking method based on the Cityscapes data set. The system generates a candidate box pool for each image and uses a neural network to score it, which uses image features, text prompts and scene context as input. We explored three model variants to study the effect of semantic guidance. The first is an ordinary candidate baseline without any scene-level constraints. The second is to limit the sampling of the candidate box to the support surface. The support surface is the area where the object can actually be placed from the segmentation diagram (for example, pedestrians can be placed on the sidewalk and cars can be placed on the road). The third one further applies the hard validity mask to remove the candidate box that violates the semantic overlap rule before scoring. Our experiments show that the support surface guidance can simultaneously improve the stability and cross-parity (IoU) of training, and the placement effect generated by hard masks is more realistic to the human eye, but may reduce IoU because some geometrically overlapping candidates are filtered out. The article discusses in detail this trade-off between metroscopic fractions and visual rationality. Code and data for replicating our experiments are available at https://github.com/QihangFeng/Object-Placement-Localization-in-Street-Scenes.

1. Introduction

Image synthesis aims to generate realistic synthetic images by inserting foreground objects into the background scene while maintaining semantic and geometric consistency. This task has a wide range of applications in image editing, content creation and data enhancement of downstream visual tasks. Recent studies have explored generating models and converter-based methods for object placement, but accurately determining the location of objects is still a fundamental challenge.

In object detection or classification, the object already exists in the image. In contrast, object placement is based on different assumptions. One of the challenges comes from the ambiguity of the placement position. A scene usually contains multiple positions, all of which seem to be semantically suitable for inserting objects, which means that there is no single correct answer. Another problem is that the object to be placed does not exist in the scene. Therefore, the model must only use context and structural clues to infer possible positions.

In order to solve these challenges, we have adopted a candidate ranking framework for object placement positioning. This method does not directly return to the boundary box coordinates, but generates a set of candidate placement areas and uses the neural scoring model to sort them. In addition, we have introduced semantic support surface guidance and hard constraint validity masks to enforce physically reasonable placement, such as placing pedestrians on the sidewalk instead of the road. The final placement position is determined by selecting the candidate area with the highest score.

2. Related work

2.1. Image composition and object placement

The problem of synthesizing the foreground object into the background scene has a long history, from the early alpha keying [1] and Poisson editing [2] to the modern learning-based process. In this field, object placement - that is, determining the position of a new object in the scene - is attracting more and more attention. The early generation method solves the placement problem through adversarial training: PlaceNet [3] generates a variety of placement positions by optimizing the loss of spatial diversity, while Compositional-GAN [4] learns the decomposition composite mapping through the appearance flow of perspective perception. GracoNet [5] adopts a different angle to model the placement into a picture completion problem based on the positive and negative composite pair of annotation. In terms of non-generative methods, TopNet [6] uses transformers to directly return to the boundary box coordinates and use sparse contrast loss, while DiffPop [7]

explores denoising diffusion to capture the scale and space relationship between objects. However, regression-based formulas tend to take their average when multiple possible positions are reasonable - this is a known problem in multimodal output tasks. We avoid this situation by sorting a group of discrete candidate objects instead of regression coordinates, and we also use the semantic a priori derived from the scene segmentation to further bias the candidate pool.

2.2. Vision-language grounding

Visual positioning - positioning the area in the image according to the free-form text description - is a related but different task. Recent visual language models, such as LLaVA [8] and Qwen-VL [9], have shown strong positioning performance through joint coding of visual and text input. From the perspective of detection, MDETR [10] integrates the text encoder with DETR through cross-attention, so that the language directly guides the regional proposal to generate, while Grounding DINO [11] further pushes it into open set detection, through text guidance Candidate. The success of these methods confirms that the "proposal-sorting" process driven by multimodal clues is a reliable way to map text to spatial areas. We draw on this high-level concept - generate candidate boxes and then score them with images, text and scene features - but our settings are fundamentally different. The positioning method is to locate the object that is already visible; we must predict the position where the missing object should appear in the repaired background, so there are no available object-level appearance clues. Since our prompt is a fixed template ("place a [category]") and there are less than ten category names, simple word embedding combined with mean pooling can capture all necessary semantics, while more complex encoders will only add parameters, but there is no obvious benefit.

2.3. BOOTPLACE

The most directly related to our work is BootPlace [12], which regards the placement of objects as placement through detection. Because the real placement of labels is sparse in nature, Zhou et al. designed a bootstrapping training strategy, which randomly removes subsets of objects from each image, thus generating up to 2^T enhancement samples for each scene, of which T is the number of marked objects. Our data construction follows the same scheme - removing objects from Cityscapes images, repairing blanks, and treating the original box as real labels. Unlike BootPlace, which implicitly discovers the placement area through the learned object query in DETR, we use semantic segmentation maps and predefined shape categories to explicitly generate candidate areas. For example, pedestrians are modeled as tall, narrow boxes, while vehicles are modeled as wide boxes. We also introduce the concept of a support surface, that is, a set of pixels on which a given object category can realistically be placed, such as a sidewalk for pedestrians or a road for vehicles, to focus sampling on reasonable areas; this is not a constraint in BootPlace. Finally, before scoring, we apply a strict validity mask to discard candidate areas that violate the semantic overlap rule. This narrows the pool of valid candidates and, as discussed in Section 4, creates a trade-off with IoU that does not exist in the original framework, but improves perceived rationality.

3. Methodology

Given a background street scene image B and a text prompt t (for example, "place a person"), our goal is to predict a boundary box to indicate the reasonable position of the specified object. Direct regression box coordinates tend to collapse to the average of all possible positions, which may cause problems when there are multiple correct positions. Therefore, we describe the task as candidate ranking: generating a set of K candidate boundary boxes $\{c_1, c_2, \ldots, c_K\}$ from the scene, and using the scoring function f_θ to evaluate each box. The final prediction is

$$\hat{b} = c_{k^*}, \text{where } k^* = \arg\max_{k \in V} f_\theta(c_k \mid B, t) \quad (1)$$

And V represents a subset of candidate objects that have passed the strict validity check derived from the scene semantic layout.

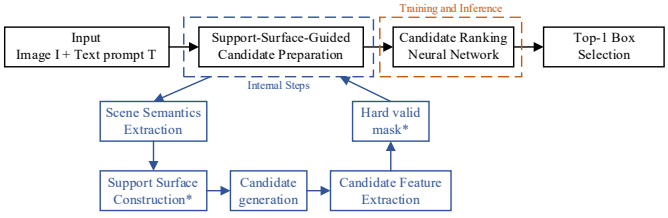

Figure 1: Overview of the proposed framework. Given a street-scene image and a text prompt, the system first extracts scene semantics and constructs a support surface to guide candidate generation. Candidate features are then extracted with a hard valid mask applied to filter implausible placements. A candidate ranking neural network scores all valid candidates, and the top-1 box is selected as the final prediction.

The system runs in four stages. First of all, we prepare the training data from the Cityscapes street view image: for each sample, remove an annotation object by repair to generate a clean background, and record its boundary box as the real location target. In the second stage, 256 candidate boxes are generated for each image in a reasonable area using semantic segmentation diagrams for sampling. In the third stage, extract multimodal features from images, text prompts, existing scene objects and each candidate box, and input them into the sorting network, which outputs a score for each candidate box. In the final stage, the unqualified candidate boxes are filtered through strict validity check, and the valid candidate box with the highest score becomes the prediction result. This design is

gradually promoted: we start with a general candidate box sorting baseline, introduce semantic a priori through support surface guidance generation, and further apply physical constraints through strict validity filtering. Each stage deals with specific defects observed in early variants.

3.1. Candidate Generation

In the basic version of our framework, the candidate box is sampled without semantic guidance, and the scoring network alone must learn which areas are suitable for placement. In fact, this is an inefficient use of the candidate budget because most randomly placed frames fall on buildings, sky or vegetation, and most of the capacity of the network is spent in an obviously impossible position of learning rejection. In order to reduce this waste, we introduced the concept of support surface: an area derived from semantics, in which the object category given in the area can actually be placed. By limiting most candidate boxes to the support surface, we focus the candidate box pool on a reasonable area and allow the sorting network to make a finer distinction between truly competitive placement positions.

Support surface definition. The support surface is a binary mask derived from the Cityscapes semantic segmentation diagram, indicating the physical location of a given object category. For pedestrians and cyclists, it corresponds to the pixel of the sidewalk. In scenes where the sidewalk area is extremely small (less than 50 pixels), such as a highway-like perspective, the mask will be expanded to include road pixels as a backup. For motor vehicles such as cars, trucks and buses, the support surface covers the pixels of roads and parking areas. Bicycles and motorcycles are handled differently, because two-wheeled vehicles appear on both roads and sidewalks in urban environments, so their support surfaces are defined as the junction of pixels of roads, sidewalks and parking areas. For any tips that do not meet the above categories, we default to the junction of roads and sidewalks as the general support area.

Bottom-center anchoring. Positioned at the bottom center. All the objects in the street scene are on a certain surface, such as pedestrians stand on the sidewalk and cars stop on the road. In order to select the candidate box, we randomly select a pixel (x_s, y_s) from the support mask, regard it as the bottom center in the normalized coordinates, and the position of the box is determined that the bottom edge is at y_s, and the horizontal center is aligned with x_s.

We have predefined different shapes for different categories to reflect their geometric differences. For pedestrians and cyclists, the scale set is (0.10, 0.14, 0.18) and the aspect ratio is (0.35, 0.45, 0.60), thus forming a high and narrow frame. For vehicles, we use scale (0.16, 0.22, 0.30) and a wider aspect ratio (1.2, 1.6, 2.0). For bicycles and motorcycles, it is somewhere between the two. The scale is (0.12, 0.16, 0.20), the aspect ratio is (0.45, 0.60, 0.75), and the ratio is (0.8, 1.1, 1.5). For 256 candidate boxes of each image, 95% comes from the supporting surface sampling. The remaining 5% is extracted from the global backup pool, which contains 225 boxes arranged in a 5×5 space grid with three scales and three aspect ratios to provide the minimum coverage outside the support surface when the semantic diagram is noisy or incomplete.

Hard valid-mask. Compared with the benchmark method, the support surface guided sampling has been significantly improved, as shown in Section 4. However, we have noticed that some forecasts still appear in suspicious positions. For example, when the prompt is "place a person", the support surface is mainly defined as a sidewalk pixel, but the sampling process may still produce some boxes that drift to adjacent roads due to the scope of the box or the inaccuracy of the semantic label boundary. The model may put this overlap with the road in the first place just because it overlaps well with the real situation geometrically, although it is unrealistic to place pedestrians in the middle of the road.

To solve this problem, we have introduced a strict effective mask to post-filter each candidate object through a series of conditions. First of all, the bottom center pixel of the candidate object must be located on the support surface - if the point where the object touches the ground is not in the walkable or driveable area, the candidate object will be rejected immediately. In addition to the inspection at this point, the area at the bottom 15% of the candidate must overlap with the support surface beyond the category-related threshold: 45% for vehicles, 35% for pedestrians, and 30% for bicycles. Finally, the semantic constraints of specific categories must be met. For vehicles, at least 20% of the candidate area must cover the road pixel, while overlap with sidewalks and people must be kept at a low level. For pedestrians, the proportion of vehicle pixels in the candidate box must be kept below 20% to prevent pedestrians from being placed in the middle of traffic.

The valid mask value of any candidate who fails to pass the condition is set to 0, and its score is set to -10^4 before the softmax during training and reasoning. In rare cases, if all candidates are rejected, keep the single candidate with the highest support ratio of the bottom strip as a backup. A direct consequence of this design is that some candidates for high crossover (IoU) - those who overlap well with the real situation but are physically unlikely - will be eliminated from the candidate pool. The model must then be selected from the remaining valid candidates. The geometric overlap of these candidates may be low, but the corresponding positions are more realistic. The tension between the accuracy of this comparison measurement and the perceived rationality is discussed in Section 4.

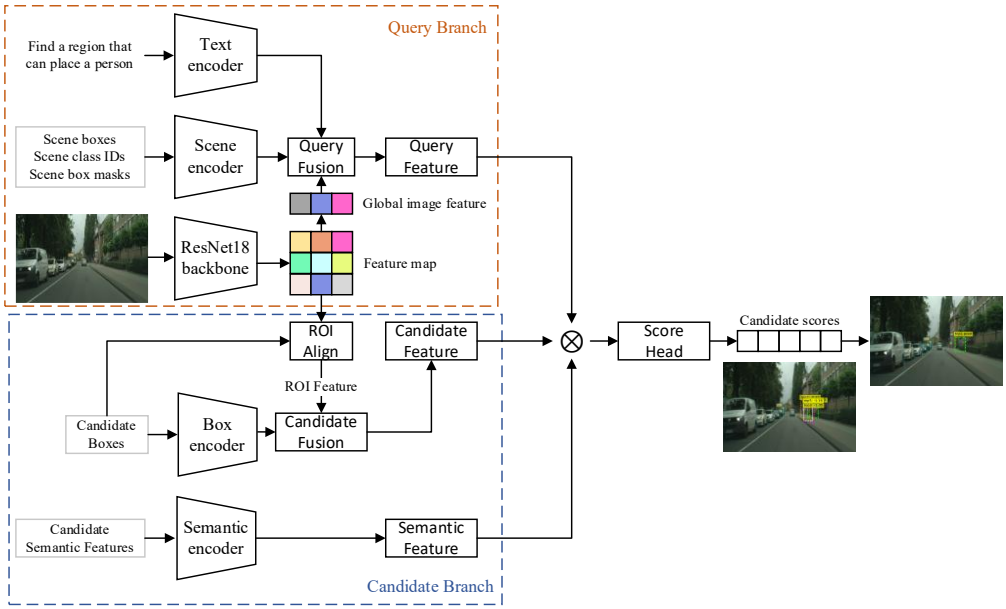

Figure 2: Architecture of the candidate ranking network. The query branch fuses text, scene context, and global image features into a shared query representation. The candidate branch encodes each candidate using ROI-aligned visual features, box coordinates, and semantic overlap ratios. A score head receives the query, candidate, and semantic features along with their interaction terms (denoted by ⊗) to produce a scalar plausibility score per candidate.

3.2. Candidate Ranking Network

The ranking network receives five types of input - background images, text prompts, scene contexts, each candidate box and the semantic combination of each candidate area, and generates a scalar plausibility score for each candidate object. During the whole training process, the image backbone network is frozen, so that the total number of trainable parameters remains at about 1.3 million.

Multi-modal feature extraction. The output of all encoders is projected to the shared 256 dimensions. Using the pre-trained weight frozen ResNet-18 as an image encoder, a 512×7×7 spatial feature map is generated from the input image of 224×224; 256-dimensional global image features are obtained through global average pooling and then through a linear layer. For text coding, each prompt mark is embedded in 128 dimensions, pool the unfilled position average, and then projected to 256 dimensions - this is a lightweight choice for our closed set prompt glossary, in which all prompts follow the pattern of "place a [category]". The scene context is captured by the boundary box and category labels of the remaining objects after coding and repair. Each object is connected by a box feature (mapped from 4-dimensional to 128-dimensional through a two-layer multi-layer perceptron) and a 32-dimensional learning category embedding, fused through a multi-layer perceptron, and then aggregated into a 256-dimensional scene vector through the mask mean pool.

For each candidate object, two parallel encoders extract complementary information. Alignment of ROI on the ResNet-18 feature map can obtain visual features from the candidate area, and a box encoder maps four normalized coordinates into geometric features. The two are connected and merged into a 256-dimensional candidate representation. In addition, a semantic encoder maps four pre-calculated overlap rates, which are roads, sidewalks, pedestrians and vehicles, to 256 dimensions through a two-layer multi-layer perceptron, so that the division can directly access the semantic combination of each candidate area.

Score head. Images, text and scene features are connected through multi-layer perceptron (MLP) projection to form a 256-dimensional query q, which is shared among all candidate objects and encodes the current scene and prompt requirements. For each candidate k, the scorer receives five components:

$$x_k = [c_k; s_k; q; c_k \odot q; s_k \odot q] \quad (2)$$

Among them, c_k is the candidate feature, s_k is the semantic feature, and $\odot$ represents element-by-element multiplication. The interaction item captures the alignment between each candidate and the query without an explicit attention mechanism. This 1280-dimensional input generates a scalar fraction for each candidate object through a multi-layer perceptron with a dropout layer ($1280 \rightarrow 256 \rightarrow 1$). In the reasoning process, an effective mask is applied, and the valid candidate with the highest

score determines the output boundary box.

3.3. Training strategy

Soft targets. For each training sample, calculate the intersection ratio (IoU) between each candidate box and the real box. Since there may be multiple correct locations, we build a soft target distribution instead of a one-hot label. The weight obtained by each valid candidate box is directly proportional to $(IoU_k \cdot m_k)^\gamma$, where m_k is the validity mask and $\gamma = 2$, and is normalized among all valid candidates:

$$p_k^{soft} = \frac{(IoU_k \cdot m_k)^\gamma}{\sum_j (IoU_j \cdot m_j)^\gamma} \quad (3)$$

Here, $m \in \{0,1\}$ is the validity mask. The exponent term makes candidates with higher IoU receive larger weights, while other overlapping candidates can still keep smaller weights. If no candidate box overlaps with the ground-truth box, the soft target reduces to a one-hot label on the best available candidate box.

Loss function. The training objective combines two terms. Soft sorting loss is the KL divergence between the soft target and the predicted score distribution, which encourages the model to sort the candidates in the order of real intersection ratio (IoU):

$$L_{soft} = -\frac{1}{B}\sum_b \sum_k p_k^{soft} \cdot log\hat{p}_k \quad (4)$$

The hard classification loss applies standard cross-entropy targeting the single best valid candidate:

$$L_{hard} = -\frac{1}{B}\sum_b log\hat{p}_{k_b^*} \quad (5)$$

The total loss is:

$$L = 1.0 \cdot L_{soft} + 0.5 \cdot L_{hard} \quad (6)$$

For soft items, the possible position in reasonable sorting reflects the multimodal characteristics of the task, while hard terms provide a steeper gradient to push the best candidate to the top of the ranking.

The batch size is 16, and the gradient norm is truncated at 1.0. In loss calculation and evaluation, they are shielded by setting the score of invalid candidates to -10^4 before softmax.

Because the hard effective mask removes candidates before scoring, the pool of valid candidates between model variants is different. We have analyzed this trade-off in Section 4.

4. Experiments and Results

4.1. Dataset and setup

Dataset Construction. Our dataset is built upon the Cityscapes benchmark, which contains urban street images annotated with both semantic labels and instance-level information. For each image, we collect existing objects along with their bounding boxes, and associate them with a text prompt that specifies the target object to insert (e.g., "place a person"). The ground-truth placement is derived from annotated object locations, following a simplified BOOTPLACE-style preprocessing pipeline. For each input image, we generate 256 candidate bounding boxes. The model is trained as a candidate ranking system rather than a direct bounding box regressor, since multiple placement locations may be plausible in the same scene.

Evaluation Metrics. We evaluate the model using three metrics: validation loss, Top-1 IoU, and Top-5 IoU. The validation loss is defined as a combination of a soft ranking loss and a hard classification loss, as we mentioned in section 3.3. The Top-1 IoU measures the overlap between the highest-scoring candidate and the ground truth, while the Top-5 IoU evaluates whether at least one of the top-ranked candidates provides a good placement.

Ablation Study. To analyze the contribution of each component, we conduct an ablation study with three model variants:

- Candidate model, a baseline without semantic constraints.
- SupportSurface model, restricting candidate generation to semantically valid regions.
- HardConstraint model, further applying a strict validity mask to enforce geometric consistency.

This design allows us to isolate the effect of semantic guidance and geometric constraints on placement performance.

4.2. Quantitative results

We quantitatively compare three variants of our method, namely Candidate, SupportSurface, and HardConstraint, using a learning rate of 1×10^{-5} over 10 epochs. Specifically, we analyze the training and validation curves of loss, Top-1 IoU, and Top-5 IoU to examine both optimization behavior and final localization performance.

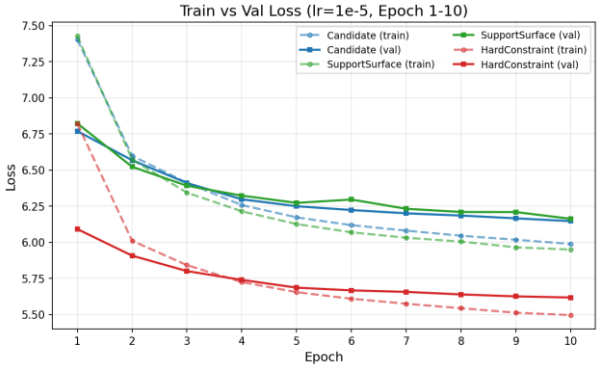

Figure 3: Comparison of training and validation loss

We first analyze the loss curves in Figure 3. All three methods exhibit smooth and stable optimization, and both training loss and validation loss decrease steadily during

the early epochs. Among the three variants, HardConstraint consistently achieves the lowest validation loss and converges to the best final validation loss. In contrast, Candidate and SupportSurface follow similar validation-loss trends and remain noticeably higher than HardConstraint throughout most of training. This indicates that the additional masking mechanism in HardConstraint makes the optimization objective easier to fit. However, lower validation loss does not directly lead to better IoU performance, which suggests that the training loss and the final localization metrics are not perfectly aligned in this task.

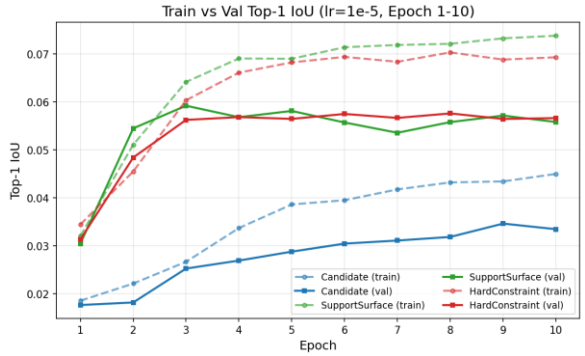

Figure 4: Comparison of training and validation Top-1 IoU

For Top-1 IoU, Figure 4 shows a more informative comparison of placement quality. The Candidate baseline improves gradually but remains clearly below the other two methods throughout training, showing that ranking candidates without semantic guidance is insufficient. The SupportSurface model improves much faster and reaches the highest validation Top-1 IoU in the early stage, demonstrating that semantic guidance during candidate generation substantially improves the quality of the top-ranked prediction. The HardConstraint model also performs much better than the Candidate baseline and remains competitive with SupportSurface, especially in the later epochs. Nevertheless, it does not provide a clear improvement over the best Top-1 IoU achieved by SupportSurface. This suggests that support-surface guidance is more effective than hard filtering for directly improving the best predicted box.

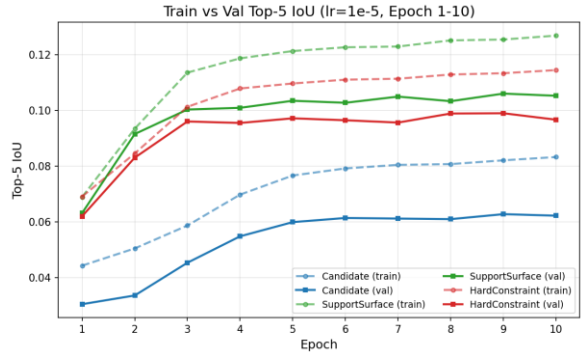

Figure 5: Comparison of training and validation Top-5 IoU

A similar trend can be observed in Figure 5, which reports Top-5 IoU. The SupportSurface model consistently achieves the best validation Top-5 IoU and maintains the strongest performance over nearly all epochs. The HardConstraint model again ranks second, clearly outperforming the Candidate baseline but still remaining below SupportSurface. Since Top-5 IoU reflects the overall quality of the top-ranked candidate set rather than only the single best prediction, this result indicates that support-surface-guided candidate generation improves not only the final selected box but also the overall candidate pool. By comparison, hard constraints help remove implausible candidates, but this stricter filtering does not further improve the diversity or IoU quality of the top candidate set.

Overall, Candidate is the weakest baseline. SupportSurface gives the best Top-1 and Top-5 IoU, showing that support-surface-guided candidate generation is the main source of quantitative improvement. HardConstraint achieves the lowest validation loss, but its IoU is still slightly below SupportSurface. This suggests that hard constraints mainly improve plausibility, while support surface contributes more to localization accuracy.

4.3. Qualitative results and Visualization

We evaluate the proposed framework from two perspectives: (1) the ability to produce prompt-conditioned placement strategies, and (2) the contribution of geometric constraints through model comparison.

Figure 6: An example of Prompt-Aware Placement (Same Image, Different Prompts)

Figure 6 shows the behavior of the SupportSurface with HardConstraint model with different prompts on the same input image. We compare the predictions for “place a car” and “place a person.”

The model produces clearly distinct placement patterns based on the object category. For the car prompt, the predicted bounding boxes are concentrated along the road surface, which is consistent with typical vehicle placement. In contrast, for “place a person.” prompt, the predictions always on the sidewalk regions, which is expected pedestrian behavior.

From these results, the model shows the capacity to capture how objects relate to their surrounding scenes. It also reflects spatial preferences that vary with the input prompt. By using support-surface guidance with hard geometric constraints together, the model is better able to rule out unsuitable regions and keep its predictions in places that make more sense for the requested object.

Figure 7: An example of Hard Constraint’s effect
(Same Prompt, Different Models)

Figure 7 presents a qualitative comparison example of the support surface model and the hard constraint model under the "place a person" instruction.

In the support surface model, candidate boxes are generated simultaneously on the roads and sidewalks. Therefore, this model sometimes directly predicts that the object will be placed on the road surface. Although in some cases these predictions may have a higher degree of coincidence with the real situation, they often appear less realistic.

In contrast, the hard constraint model is more inclined to place the object on the sidewalk, which is closer to the position where pedestrians usually appear. Even if the IoU score is slightly lower, these placement methods often seem more reasonable at the semantic level.

Overall, this indicates that adding physical constraints and scene-level information helps to improve the realism of the final placement effect.

4.4. Failure cases and analysis

We analyzed the failure cases to better understand the limitations of the proposed framework. Figure 8 provides an example where the predicted placement location overlaps with the existing objects in the scene.

In this example, the model received the prompt "Place a car". The predicted bounding box is above the road surface, which is semantically reasonable. However, the predicted placement location overlaps with the existing vehicle, which is an unrealistic result.

Figure 8: An example of the failure case

This behavior seems to come from how the model interprets the scene. It relies entirely on 2D appearance and semantic labels, without any notion of depth. Because of that, it treats all visible regions in a similar way and has no reliable way to tell whether a space is actually free.

This becomes more noticeable when considering how 3D structure is projected into an image. A region that looks occupied in 2D may still be available in the real scene, or the opposite may happen due to perspective. Without access to depth cues, the model cannot resolve this ambiguity. Similar issues have been mentioned in earlier work, so this is not unique to our setup. In practice, this limitation can lead to overlaps or placements that look inconsistent with the scene.

Overall, this example points to a missing piece in the current design. Incorporating some form of geometric reasoning or depth information could help the model better judge whether a region is already occupied and reduce these types of errors.

5. Conclusion, limitation and future work

In this work, we focus on a candidate ranking approach for placing objects in street scenes. Candidate generation is guided by support surfaces, and a hard validity constraint is applied to filter out clearly unsuitable locations. This combination reduces the search space and tends to produce placements that better match the surrounding scene. In our experiments, the model also shows more stable training behavior and produces results that look more reasonable compared with the baseline.

However, a few limitations that become apparent when looking more closely at the results. One of them is the reliance on IoU for evaluation. The IoU mainly reflects the geometric overlap situation. In cases where there are

multiple possible placements and all of these placements can be considered reasonable, it does not always align with the seemingly reasonable state in the scene. Another issue is related to the way the dataset is constructed. Since ground-truth positions are obtained by removing objects from images, small artifacts or inconsistencies can remain. The model may then pick up on these cues instead of learning more general placement behavior, which makes it harder to tell what it has actually learned.

These points suggest several possible directions for our future work. For evaluation, it may be useful to find other metrics that go beyond IoU and better reflect visual or semantic quality. In terms of data, exploring different construction strategies - such as using synthetic data or more carefully controlled scene editing - can help reduce bias and improve generalization ability. In addition, incorporating depth information or 3D-aware representations may help the model better judge whether a region is already occupied and reduce unrealistic placements.

6. Team contributions and repository

Zixi Chai developed the candidate preparation pipeline, including support-surface-guided candidate generation and hard constraints for plausibility control.

Qihang Feng developed the ranking neural network, including the query branch, candidate branch, and score head for candidate ranking.

Hongtao Ren designed the evaluation method and analysis, including quantitative comparison across model variants and interpretation of the results.

The implementation repository for this project, including the code and related resources, is available at: https://github.com/QihangFeng/Object-Placement-Localization-in-Street-Scenes/

Acknowledgements

We would like to thank the authors of BOOTPLACE for their inspiring ideas and valuable contributions to this research direction. Their work provided an important starting point for our project. We also sincerely thank our course instructor for the guidance and feedback throughout this project. Finally, we are grateful to the open-source community for providing useful resources, articles, and tools that supported our implementation and experiments.

References

[1] Wang J, Cohen M F. Image and video matting: a survey. Foundations and Trends in Computer Graphics and Vision, 2007.

[2] Pérez P, Gangnet M, Blake A. Poisson image editing. ACM SIGGRAPH, 2003.

[3] Zhang Z, Zhao Z, Lin Z. PlaceNet: Neural spatial representation learning for object placement. ECCV, 2020.

[4] Azadi S, Pathak D, Ebrahimi S, Darrell T. Compositional GAN: Learning image-conditioned binary composition. IJCV, 2020.

[5] Zhou H, Chen C, Li C, et al. GracoNet: Object placement via graph completion. ECCV, 2022.

[6] Zhou H, Li C, Zuo X, et al. TopNet: Transformer-based object placement network for image compositing. CVPR, 2023.

[7] Liu Z, Zhou H, Li C, et al. DiffPop: Plausibility-guided diffusion for multi-object placement. arXiv preprint, 2024.

[8] Liu H, Li C, Wu Q, Lee Y J. Visual instruction tuning. NeurIPS, 2023.

[9] Bai J, Bai S, Yang S, et al. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. arXiv preprint, 2023.

[10] Kamath A, Singh M, LeCun Y, et al. MDETR: Modulated detection for end-to-end multi-modal understanding. ICCV, 2021.

[11] Liu S, Zeng Z, Ren T, et al. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. ECCV, 2024.

[12] Zhou H, Zuo X, Ma R, Cheng L. BOOTPLACE: Bootstrapped object placement with detection transformers. CVPR, 2025.